DIY 2 — Militarized interstate disputes as a relational event model

remverse hands-on: tie & actor models, moving windows, event types, and duration

Relational Event Modeling workshop

2026-07-29

How to use this document. Paste the blocks into R and run in order. Chunks are not executed while knitting (`eval = FALSE`); expected results are described in the text. The tie- and actor-oriented fits are fast; the moving-window loop refits the model several times and takes a minute or two.

Introduction and learning objectives

The data are **militarized interstate disputes (MIDs)** from the Correlates of War project: a directed event $i \to j$ means state i initiated a militarized action against state j. We use a coherent subset — **Middle-Eastern states plus the intervening major powers** (Israel, Iraq, Iran, Syria, Egypt, Turkey, Jordan, Saudi Arabia, … + USA, Russia, UK, France): **19 actors, 881 events, 1817–2001**.

The aim of the tutorial is to

- put event times **and** time-varying covariates on **one common clock**;
- fit a **tie-oriented** REM with a rich mix of endogenous and exogenous effects;
- run a **moving window** to check whether effects are *constant over time* (the REM-native proportional-hazards / Schoenfeld check);
- fit an **actor-oriented** model (who initiates, and whom do they target);
- use the **hostility level** as an **event type** (`extend_riskset_by_type`, `consider_type`) to model **escalation**; and
- see how events with **duration** (disputes have start *and* end dates) extend the framework.

Setup: packages and data

```r
# install.packages("remverse")
library("remverse")
```

```r
data_dir <- "workshop_data/military"
cov <- function(f) read.csv(file.path(data_dir, f), stringsAsFactors = FALSE)

el <- cov("edgelist_subset.csv")      # 881 events; use edgelist_full.csv for all 4,670
el$time     <- as.Date(el$time)
el$end_date <- as.Date(el$end_date)
el <- el[order(el$time), ]
head(el)
```

One clock for everything

remstats looks up each covariate at the value **in force at the event time**, so events and covariates must live on the **same numeric time scale**. We build that scale ourselves — **decimal years since the first event** — and reuse it for every covariate. (Doing it explicitly is clearer than relying on internal conversions, and it *is* the "effective time" concept from the lecture.)

```
origin   <- min(el$time)                                   # first event = time 0
to_years <- function(d) as.numeric(as.Date(d) - origin) / 365.25
el$t     <- to_years(el$time)

# helper: make a symmetric dyadic covariate directed both ways, then de-duplicate
symmetrize <- function(d) {
  r <- d; r$actor1 <- d$actor2; r$actor2 <- d$actor1
  unique(rbind(d, r))
}
```

The relational event history

Tie-oriented, interval timing, **active-saturated** risk set (every ordered pair of the 19 observed states is at risk at every time point):

```
reh <- remify(el[, c("time", "actor1", "actor2")],
              directed = TRUE, model = "tie", riskset = "active_saturated",
              time_units = "years")
reh
plot(reh)
```

Covariates on the common clock

Actor-level covariates need columns `name`, `time`, value; dyadic covariates need `actor1`, `actor2`, `time`, value. Convert each file's `date` with `to_years()`:

```
# --- actor-level (time-varying) ---------------------------------------------
cinc <- cov("cov_cinc.csv");      cinc$time <- to_years(cinc$date)   # capabilities (CINC)
cinc <- cinc[, c("name", "time", "cinc")]

dem  <- cov("cov_dem.csv");       dem$time  <- to_years(dem$date)    # Polity score
dem  <- dem[, c("name", "time", "dem")]

rel  <- cov("cov_religion.csv");  rel$time  <- to_years(rel$date)    # religion category
rel  <- rel[, c("name", "time", "religion_id")]

milper <- cov("cov_milper.csv");  milper$time <- to_years(milper$date)  # military personnel
milper <- milper[, c("name", "time", "milper")]

# --- dyadic (time-varying), symmetric ---------------------------------------
alliance <- cov("cov_alliance.csv"); alliance$time <- to_years(alliance$date)
alliance <- symmetrize(alliance[, c("actor1", "actor2", "time", "alliance")])

contig <- cov("cov_contiguity.csv"); contig$time <- to_years(contig$date)
contig <- symmetrize(contig[, c("actor1", "actor2", "time", "contiguity")])
```

> **Coverage tip.** `remstats` needs a covariate value at or before an actor's first appearance. The COW files start in 1816, so all 19 actors are covered. If you switch to a different subset and hit a coverage error, either restrict to covered actors or drop the offending effect.

Tie-oriented model: the four-step workflow

Steps 1–3 — effects, statistics, estimation

Endogenous **mechanisms** + exogenous **who-fights-whom** effects + one interaction:

```
tie_effects <- ~
  inertia(scaling = "std") +                           # repeat disputes with the same target
  reciprocity(scaling = "std") +                       # fight back
  otp(scaling = "std") +                               # transitivity: enemy-of-my-enemy structure
  send("cinc",    attr_actors = cinc) +                # powerful states initiate more?
  receive("cinc", attr_actors = cinc) +                # weak states targeted more?
  difference("dem", attr_actors = dem) +               # regime-type distance drives conflict?
  same("religion_id", attr_actors = rel) +             # shared religion dampens conflict?
  tie("alliance",   attr_dyads = alliance) +           # do allies fight less?
  tie("contiguity", attr_dyads = contig) +             # do neighbours fight more?
  inertia(scaling = "std"):reciprocity(scaling = "std")   # do the two reinforce?

stats <- remstats(reh, tie_effects = tie_effects, first = 50)
fit   <- remstimate(reh, stats)
summary(fit)
```

Interpretation runs through the rate: a positive `tie_contiguity` means bordering states produce disputes at a higher rate; a negative `tie_alliance` means allied dyads are slower to fight; `receive_cinc` < 0 with `send_cinc` > 0 would say strong states *start* disputes but are *targeted* less.

Step 4 — diagnostics

```
diag <- diagnostics(fit, reh, stats)
print(diag)
plot(diag)
head(diag$surprise_offenders)   # the disputes the model least expected
```

Are the effects constant over time? A moving window

An interval tie-REM is a **proportional-hazards** model: it assumes each coefficient is **constant across the whole 1817–2001 span**. In survival analysis you would test this with **Schoenfeld residuals**; the REM-native equivalent is to **re-estimate along the stream** and look for drift.

```
rw <- remwindow(reh, stats, window.width = 300, step.size.window = 100)
plot(rw)        # each coefficient vs. the window mid-point
summary(rw)
```

Read the bands: a **flat** trajectory supports the constant-effect assumption; a **drift or jump** means the mechanism itself changed — e.g. contiguity or reciprocity behaving differently across the Cold War and post-1990 periods. Because the sample size is the number of *events*, this is a genuine time-series of evidence, not an artifact.

Actor-oriented model: initiation vs. target choice

Separate the two decisions: **who acts next** (a sender-rate model) and, given the sender, **whom do they target** (a receiver-choice model).

```
reh_ao <- remify(el[, c("t", "actor1", "actor2")],
                 directed = TRUE, model = "actor", riskset = "active_saturated")

sender_effects <- ~                                  # who starts a dispute?
  send("cinc",   attr_actors = cinc) +              #   powerful states initiate
  send("milper", attr_actors = milper, scaling = "std") +   # larger militaries initiate
  indegreeSender(scaling = "std")                    #   frequently-targeted states hit back

receiver_effects <- ~                                # given a sender, whom?
  inertia(scaling = "std") +
  reciprocity(scaling = "std") +
  receive("cinc",   attr_actors = cinc) +
  difference("dem", attr_actors = dem) +
  tie("contiguity", attr_dyads = contig) +          # neighbours are targeted
  same("religion_id", attr_actors = rel)

stats_ao <- remstats(reh_ao,
                     sender_effects   = sender_effects,
                     receiver_effects = receiver_effects)
fit_ao   <- remstimate(reh_ao, stats_ao, method = "MLE")
summary(fit_ao)                                      # two coefficient tables
```

The split often clarifies theory: capability may drive *initiation* while proximity drives *target choice*.

Event types: modeling escalation with hostility level

MID events carry a hostility level `hihost` on a 2–5 ladder (2 = threat, 3 = display, 4 = use of force, 5 = war). Treating it as an **event type** lets the history operate *within* or *across* rungs — the natural way to ask about **escalation**.

```
reh_ty <- remify(cbind(el[, c("t", "actor1", "actor2")], hihost = ifelse(el$hihost<=3,"low","high")),
                 directed = TRUE, model = "tie",
                 event_type = "hihost",
                 riskset = "active_saturated",
                 extend_riskset_by_type = TRUE)   # each dyad x level is its own unit at risk
reh_ty
head(reh_ty$edgelist)
# consider_type: "separate" = one inertia per level; "interact" = effect depends
# on the CURRENT level -> does a history of threats predict the next war?
stats_ty <- remstats(reh_ty,
                     tie_effects = ~ inertia(consider_type = "interact", scaling = "std") +
                                     reciprocity(consider_type = "interact", scaling = "std"))
stats_ty
fit_ty   <- remstimate(reh_ty, stats_ty, method = "MLE")
summary(fit_ty)
```

`consider_type = "interact"` is where escalation lives: a positive interaction of past lower-level hostility on higher-level events is evidence that disputes *ratchet up* between the same pair.

`FEtype()` is the lighter alternative — a separate baseline rate per hostility level, without changing how the endogenous effects work. Swap it in when you only want type-specific base rates.

The same variable as a *weight* instead

`hihost` can equally serve as an **event weight** — one process, but more intense events count more toward the endogenous statistics:

```
el$weight <- as.numeric(el$hihost)                    # 2..5
reh_w <- remify(el[, c("time","actor1","actor2","hihost")], event_weight = "hihost",
                directed = TRUE, model = "tie", riskset = "active_saturated")
print(reh_w)
```

Subsequently, the same models as above could be fitted and assessed.

Type or weight is a modeling choice: a *type* splits the stream into kinds of event (escalation *between* kinds); a *weight* keeps one stream and scales intensity. Let your research question decide.

Extension: modeling event duration

Every event so far is an instant. But disputes **last** — the edge list carries an `end_date`, so each event has a **duration**:

```
event_duration <- as.numeric(el$end_date - el$time)/365   # dispute length in years
summary(event_duration)
hist(event_duration, breaks = 40, main = "Dispute durations (years)", xlab = "years")
```

Duration separates two questions the instantaneous REM merges:

1. **Onset** — when does the *next* dispute start? → the standard REM above, modeling the rate on `time`.
2. **Duration** — once started, how long until it *ends*? → a companion survival model with `duration` as the outcome and the same covariates as predictors (hostility, capability, alliance, . . .).

A RuREM analysis can be performed in a similar fashion using `remverse` as above. The end time of an event needs to have column name `end_time`, another option is have have a column name named `duration` containing the duration of events. For the MID data, we assume that the initiator of the dispute is also the ender of the dispute; thus, we assume a directed end of events.

```
colnames(el)[4] <- "end_time"
reh_dur <- remify(el[, c("time", "actor1", "actor2", "end_time")],
           directed = TRUE, model = "tie", riskset = "active_saturated",
           duration = TRUE, dur_directed_end = TRUE, time_units = "years")
reh_dur
plot(reh_dur)
```

We specify models for starting and ending a dispute.

```
start_effects <- ~
  inertia(scaling = "std") +                          # repeat disputes with the same target
  reciprocity(scaling = "std") +                      # fight back
  otp(scaling = "std") +                              # transitivity: enemy-of-my-enemy structure
  difference("dem", attr_actors = dem) +              # regime-type distance drives conflict?
  tie("contiguity", attr_dyads = contig)              # do neighbours fight more?
end_effects <- ~
  inertia(scaling = "std") +                          # repeat disputes with the same target
  reciprocity(scaling = "std") +                      # fight back
```

```r
  otp(scaling = "std") +                             # transitivity: enemy-of-my-enemy structure
  difference("dem", attr_actors = dem) +             # regime-type distance drives conflict?
  tie("contiguity", attr_dyads = contig)             # do neighbours fight more?

stats_dur <- remstats(reh_dur, start_effects = start_effects,
                  end_effects = end_effects,
                  first = 50)
fit_dur   <- remstimate(reh_dur, stats_dur)
summary(fit_dur)
```

The results show how the past affects the rates at which military disputes start and end. The positive `inertia` effect on starting suggests a country is more likely to initiate a dispute against a target it has attacked before. The negative `reciprocity` effect on ending suggests that a country that has often been attacked by another is slower to end the ongoing dispute. The positive `tie_contiguity` effect on starting indicates that neighboring countries are more likely to engage in disputes than non-neighboring ones, while its negative effect on ending indicates that disputes between neighbors tend to last longer. These are effects on the rate of starting and ending; as the diagnostics below show, several are clearly significant yet add little predictive discrimination — significance and recall answer different questions.

Finally, we check some diagnostics to assess the fit of the DuREM:

```r
diag_dur <- diagnostics(fit_dur,reh_dur,stats_dur)
print(diag_dur)
plot(diag_dur)
```

The joint recall (median rank 0.94, prob ratio 9.2) should be interpreted with caution. The joint risk set at each time point combines the dormant dyads that could start a dispute with the ongoing disputes that could end. Because an ongoing dispute has a higher baseline hazard of ending than a dormant dyad has of starting (compare `baseline.end` and `baseline.start`), ending events tend to receive high ranks regardless of the exact choice of the `end_effects`. The separate start and end recall are more informative, since each event is ranked only within its own risk set. The start recall (prob ratio 1.6) indicates a modest effect of the covariates on which dyad initiates a dispute, while the end recall (prob ratio 1.1) is close to chance, suggesting that the covariates provide little information about which ongoing dispute is likely to end next.

Exercises

1. **Compare models.** Consider one of the above example analyses. Try different models, e.g., models with additional sets of effects, or models which only contain the endogenous effects, or only the exogenous effects. Compare model fit.
2. **Risk set.** Try different risksets and check how this affects the results.
3. **Memory.** The `full` memory was used implyig equal weight for all past events regardless of their transpired times. Is this realistic? Instead try an exponential `decay` function for the memory retention. Try different half-life parameters and compare their model fit.
4. **Memory.** The duration parameters `psi_start` and `psi_end` are set to 1 by default. This implies a linear increase of the weight as function of the duration of past events. Try different duration parameters and compare their fit.
5. **Full data.** Instead of working with the subset `edgelist_subset.csv`, try the full MID data set `edgelist_full.csv` (4,670 events, all states). Try to different REMs for the full sequence.

References

Butts, C. T. (2008). A relational event framework for social action. *Sociological Methodology, 38*(1), 155–200.

Lakdawala, R., Leenders, R., Ejbye-Ernst, & Mulder, J. A joint model for the occurrence and duration of relational events.

Lakdawala, R., Leenders, R. , & Mulder, J. (2026). Not all bonds are created equal: dyadic latent class models for relational event data. *Social Networks, 87*, 135–151.

Meijerink-Bosman, M., Back, M., Geukes, K., Leenders, R., & Mulder, J. (2023). Discovering trends of social interaction behavior over time: An introduction to relational event modeling. *Behavior Research Methods, 55*, 997–1023.

Singer, J. D. (1972). The "Correlates of War" project: Interim report and rationale. *World Politics, 24*(2), 243–270.

Stadtfeld, C., & Block, P. (2017). Interactions, actors, and time: Dynamic network actor models for relational events. *Sociological Science, 4*, 318–352.